

What Is Query Processing?

When you write a query in SQL, for example:

```
SELECT name FROM instructor WHERE salary > 50000;
```

You're asking the Database Management System (DBMS) to give you results.

But before showing results, the DBMS must process this query in several steps — like how a chef follows steps to cook food.

Steps of Query Processing

1. Parsing and Translation
2. Optimization
3. Evaluation

Let's understand each one clearly.

Step 1: Parsing and Translation

When you write an SQL query, DBMS first parses it:

- It checks if the syntax is correct (like grammar in English).
- It verifies that the tables and columns exist.
- Then it translates the query into an internal form, usually Relational Algebra (a kind of mathematical expression for databases).

Example:

SQL Query:

```
SELECT id, salary
```

```
FROM instructor
```

```
WHERE id > 50501;
```

Relational Algebra:

```
 $\sigma_{id > 50501} (\pi_{id, salary} (instructor))$ 
```

Here:

- σ (sigma) means Selection (choose rows).
 - π (pi) means Projection (choose columns).
 - instructor is the table (relation).
-

Step 2: Optimization

The DBMS can execute the same query in many ways.

For example:

1. First choose columns, then select rows →
 $\sigma_{id > 50501} (\pi_{id, salary} (instructor))$
2. Or first select rows, then choose columns →
 $\pi_{id, salary} (\sigma_{id > 50501} (instructor))$

Both give the same answer! 

But one may be faster — depending on how the data is stored (sorted, indexed, etc.)

The query optimizer chooses the lowest-cost way.

Step 3: Evaluation

Finally, the query execution engine takes the chosen evaluation plan and runs it to produce the result.

Measures of Query Cost

To choose the best plan, the DBMS estimates cost.

The main cost is disk access — reading/writing data.

Why?

Because reading from disk is much slower than CPU operations.

Three main disk costs:

1. Number of seeks (S)
→ How many times the disk head moves to a new track.
 2. Number of blocks read (b)
→ How many blocks are read from disk to memory.
 3. Number of blocks written
→ How many blocks are written back to disk.
-

Total Disk Cost Formula

$$\text{Cost} = (b \times t_T) + (S \times t_S) \quad \text{Cost} = (b \times t_T) + (S \times t_S)$$

where

- t_T = time to transfer one block
- t_S = time for one seek

We're using the cost formula:

$$\text{Total Cost} = (b \times t_T) + (S \times t_S) \quad \text{Total Cost} = (b \times t_T) + (S \times t_S)$$

where

- $b = 100$ blocks (number of block transfers)
 - $S = 10$ seeks
 - $t_T = 0.1$ ms per block transfer
 - $t_S = 4$ ms per seek
-

Now substitute:

$$\text{Cost} = (100 \times 0.1) + (10 \times 4) \quad \text{Cost} = (100 \times 0.1) + (10 \times 4)$$

Compute each part:

$$100 \times 0.1 = 10 \quad 10 \times 4 = 40$$

Add them together:

$$10 + 40 = 50 \text{ ms}$$

✓ Correct total cost = 50 ms, not 41 ms.

What Is Selection?

Selection means choosing **some rows** from a table that satisfy a given condition.

Think of it like **filtering** data.

Example

Let's take a simple table:

id	name	salary
101	Asha	40000
102	Rahim	55000
103	Rafi	60000

Now, we write this SQL query:

```
SELECT * FROM instructor WHERE salary > 50000;
```

👉 This means: "Give me all rows where salary > 50000."

So the **output** will be:

id	name	salary
102	Rahim	55000

id name salary

103 Rafi 60000

In Relational Algebra

Selection is written using the **sigma symbol** (σ):

$\sigma_{\text{salary} > 50000}(\text{instructor})$ $\sigma_{\{\text{salary} > 50000\}}(\text{instructor})$ $\sigma_{\text{salary} > 50000}(\text{instructor})$

This means **select all tuples (rows)** from instructor where salary > 50000.

How DBMS Performs Selection

DBMS can perform selection in different ways.

There's not just *one* method — it depends on:

- whether the table is sorted,
- whether there's an index,
- or how big the table is.

Let's look at the main algorithms.

Algorithm A1: Linear Search (File Scan)

This is the **simplest and most general** method.

How it works

1. Read each block of the file from disk (one by one).
2. For each record, check if it satisfies the condition (e.g., salary > 50000).
3. If yes, keep it in the result.
4. Continue until all blocks are checked.

Advantages:

- Works on any table.
- Works even if there's **no index**.

Disadvantages:

- **Very slow** if the table is big (you must check every row).
-

Cost Calculation for Linear Search

Let:

- b_r = number of blocks in relation r
- 1 seek is needed to start reading
- each block is transferred once

Then:

$$\text{Cost} = b_r \times t_T + 1 \times t_S \quad \text{Cost} = b_r \times t_T + 1 \times t_S$$

Example

Say the table has:

- $b_r = 200$ blocks
- $t_T = 0.1$ ms per block
- $t_S = 4$ ms per seek

$$\text{Cost} = (200 \times 0.1) + 4 = 20 + 4 = 24 \text{ ms} \quad \text{Cost} = (200 \times 0.1) + 4 = 20 + 4 = 24 \text{ ms}$$

 So total cost = **24 ms**

Best Case

If selection is on a **key attribute** (like id), and you stop as soon as you find the record, then you only need to search half the blocks on average:

$$\text{Cost}(\text{best}) = b_r \times t_T + 1 \times t_S \quad \text{Cost}(\text{best}) = \frac{b_r}{2} \times t_T + 1 \times t_S$$

Example:

$$= (100 \times 0.1) + 4 = 10 + 4 = 14 \text{ ms} \quad = (100 \times 0.1) + 4 = 10 + 4 = 14 \text{ ms}$$

✿ Algorithm A2: Primary Index (Equality on Key)

If the column you are searching is **indexed**, DBMS doesn't need to scan the whole table!

It can **jump directly** to the record using the index.

■ Example

SELECT * FROM instructor WHERE id = 103;

If there's an index on id, DBMS can use that.

Let:

- h_i = height of the index tree
- Usually 2 or 3 (small number)
- Each level requires a **seek** and **block transfer**

📊 Cost Formula

$$\text{Cost} = (h_i + 1) \times (t_S + t_T) \quad \text{Cost} = (h_i + 1) \times (t_S + t_T)$$

📊 Example

Say:

- $h_i = 3$
- $t_S = 4 \text{ ms}$
- $t_T = 0.1 \text{ ms}$

Then:

$$\text{Cost} = (3 + 1) \times (4 + 0.1) = 4 \times 4.1 = 16.4 \text{ ms} \quad \text{Cost} = (3 + 1) \times (4 + 0.1) = 4 \times 4.1 = 16.4 \text{ ms}$$

✓ Much faster than 24 ms (linear search)!

✿ Algorithm A3: Primary Index (Equality on Non-Key)

If the search key is not unique (like multiple people with the same dept-name), the DBMS:

1. Finds the **starting point** using the index.
2. Then reads all **consecutive blocks** that match.

If bbb = number of blocks containing matching records:

$$\text{Cost} = h_i(t_T + t_S) + t_S + b \times t_T \quad \text{Cost} = h_i(t_T + t_S) + t_S + b \times t_T$$

✿ Algorithm A4: Secondary Index (Equality on Non-Key)

If the index is **secondary** (not based on table's order), matching records may be on **different blocks**, so DBMS must perform **many seeks**.

If nnn = number of matching records:

$$\text{Cost} = (h_i + n) \times (t_T + t_S) \quad \text{Cost} = (h_i + n) \times (t_T + t_S)$$

⚠ Can be **very expensive** if many matches!

🧠 Summary Table

Algorithm Type		Works When	Cost (approx)	Notes
A1	Linear Search	Any table	$b_r t_T + t_S$	Slow but general

Algorithm Type	Works When	Cost (approx)	Notes
A2 Primary Index, equality on key	Index exists	$(h_i+1)(t_T+t_S)(h_i+1)(t_T+t_S)$	Fastest
A3 Primary Index, non-key	Index + multiple records	$h_i(t_T+t_S)+t_S+bt_{hi}(t_T+t_S) + t_S + bt_{hi}(t_T+t_S)+t_S+bt_T$	Moderate
A4 Secondary Index, non-key	Secondary index	$(h_i+n)(t_T+t_S)(h_i+n)(t_T+t_S)(h_i+n)(t_T+t_S)$	Expensive

✅ In short:

- If you have an **index**, DBMS uses it → very fast.
- If not, DBMS uses **linear scan** → slow but guaranteed to work.

✖ A5: Primary Index, Comparison Condition

This is used when we have a **primary index** (the table is **physically sorted** on the indexed attribute), and the condition is a **range** or **comparison**, such as:

$\sigma_{id > 50501}(\text{Instructor})$

⚙ How It Works

1. Use the **index** to find the **first tuple** satisfying the condition.
(e.g., first record where id = 50502 if sorted by id)
2. Then **sequentially scan** the rest of the table (since it's sorted).

🎨 Example

Suppose instructor table is sorted by id, and we run:

SELECT * FROM instructor WHERE id >= 50501;

Then:

- Use the index to **find the first matching record (50501)**.
- **Read sequentially** from that point to the end of the table.

🎨 Cost Formula

Let:

- h_i = height of index tree
- t_T = transfer time per block
- t_S = seek time per block
- b = number of blocks to read after starting point

Then:

$\text{Cost} = h_i(t_T+t_S) + b \times t_T$

✅ Much cheaper than a full table scan (A1), because you skip directly to the first matching record!

🧠 Notes:

- Works **only if** the table is physically sorted on that attribute.
- Very efficient for **range queries** like >, <, >=, <=.

✖ A6: Secondary Index, Comparison Condition

If you have a **secondary index** (not sorted in that order),

you **can't scan sequentially** after the first match — because records are scattered all over the disk.

⚙️ Example

SELECT * FROM instructor WHERE salary > 50000;
If salary has a **secondary index**,
then all matching records are spread across the file.

📊 Cost Formula

Let:

- h_{ih_i} = index height
- n_{nn} = number of matching records

Then:

$$\text{Cost} = h_i(t_T + t_S) + n(t_T + t_S)$$

Each record might be on a **different block**,
so this can be **very expensive** (many seeks).

⚠️ Summary

Type	Sorted?	Efficient for	Comment
A5	Yes (Primary)	Range queries	Fast
A6	No (Secondary)	Range queries	Expensive, many seeks

✖️ A7: Conjunctive Selection Using One Index

Now we move to **complex selections** — where you have **multiple conditions joined by AND (∧)**.

📊 Example

SELECT * FROM instructor
WHERE dept_name = 'CSE' AND salary > 50000;
Relational algebra:
 $\sigma_{dept_name='CSE' \wedge salary > 50000}(Instructor)$
 $\sigma_{dept_name='CSE'}(Instructor) \bowtie \sigma_{salary > 50000}(Instructor)$

⚙️ How It Works

1. If there are **indices** on some attributes (say dept_name or salary), choose **one** index that gives the **lowest cost**.
 2. Use that index to fetch the matching records.
 3. Once those tuples are fetched into memory, test the **other conditions** (without indices) **in memory**.
-

📊 Cost Formula (Approximation)

The cost is the same as the **best index** you choose,
plus some **CPU time** for checking other conditions in memory (often ignored in disk cost).

🧠 Example

If:

- dept_name has an index → cost = 50 ms
- salary has an index → cost = 70 ms

→ DBMS will choose the **dept_name index** because it's cheaper,
then filter further by salary > 50000 in memory.

✅ Much more efficient than scanning both fully.

🌟 A10: Disjunctive Selection (OR Conditions)

Now we move to **OR (V)** conditions.

📺 Example

```
SELECT * FROM instructor
```

```
WHERE dept_name = 'CSE' OR salary > 50000;
```

Relational algebra:

$$\sigma_{dept_name='CSE' \vee salary > 50000}(Instructor) \setminus \sigma_{dept_name='CSE'} \setminus \sigma_{salary > 50000}(Instructor)$$

⚙️ How It Works

If **all conditions** have available **indices**:

1. Use the corresponding index for each condition to get a **set of record pointers**.
 - One for dept_name = 'CSE'
 - One for salary > 50000
2. Take the **union** of all these sets (so no duplicates).
3. Fetch those records from the file.

📺 Cost Formula (Approximation)

Cost = Sum of costs for each index + Cost to fetch all unique records
$$\text{Cost} = \text{Sum of costs for each index} + \text{Cost to fetch all unique records}$$

⚠️ If No Indices Are Available

→ Fall back to **A1 (Linear Scan)**

🧠 Example

If:

- Index on dept_name → cost = 40 ms
- Index on salary → cost = 60 ms

→ Use both indices, union results.

If no index existed → scan all rows (slowest).

📄 Summary of A1–A10 (Simplified)

Algo	Description	Works When	Main Cost Idea
A1	Linear search	Always	$b_r t_T + t_{S_b_r} t_T + t_{S_b_r} t_T + t_S$
A2	Primary index, equality on key	Unique key	$(h_i + 1)(t_S + t_T)(h_i + 1)(t_S + t_T)(h_i + 1)(t_S + t_T)$
A3	Primary index, equality on non-key	Multiple matches	$h_i(t_T + t_S) + t_S + b_{t_{h_i}}(t_T + t_S) + t_S + b_{t_{h_i}}(t_T + t_S) + t_S + b_{t_{h_i}}(t_T + t_S)$
A4	Secondary index, equality on non-key	Not sorted	$(h_i + n)(t_T + t_S)(h_i + n)(t_T + t_S)(h_i + n)(t_T + t_S)$
A5	Primary index, comparison	Sorted	$h_i(t_T + t_S) + b_{t_{h_i}}(t_T + t_S) + b_{t_{h_i}}(t_T + t_S) + b_{t_{h_i}}(t_T + t_S)$
A6	Secondary index, comparison	Not sorted	$h_i(t_T + t_S) + n(t_T + t_S)h_i(t_T + t_S) + n(t_T + t_S)h_i(t_T + t_S) + n(t_T + t_S)$
A7	Conjunctive (AND) selection	Multiple conditions	Choose best index; test rest in memory
A10	Disjunctive (OR) selection	Multiple conditions	Use all indices, union results

✅ In summary:

- **A1–A4:** Simple single-condition selections
- **A5–A6:** Range selections (comparisons)
- **A7:** AND conditions (use one best index)
- **A10:** OR conditions (use multiple indices, then union)

🧠 Why Sorting Is Needed

Sorting is used in many database operations, such as:

- ORDER BY in SQL
- GROUP BY
- DISTINCT
- JOIN (some join algorithms require sorted data)

⚙️ Problem: Large Data

If your table is **small** (fits in memory), sorting is easy — DBMS can use **quicksort** or **merge sort** in memory. But if your table is **large** (does not fit in memory), you can't load it all at once.

👉 You must sort it **in parts** — that's where **External Sorting** is used.

🔗 What Is External Sort-Merge?

It's a method that sorts data stored on disk using a **limited memory** (RAM).

It happens in two main phases:

1. **Run Creation**
2. **Merge Phase**

Let's break both down simply.

📅 Example Setup

Let:

- Relation (table) size = $b_r = 100$ blocks
- Memory size = $M = 3$ blocks

That means we can load **only 3 blocks at a time** into memory.

📅 Step 1: Run Creation Phase

We divide the table into chunks that **fit in memory**, sort each chunk, and save it back to disk.

Steps:

1. Read **M blocks (3 blocks)** from disk into memory.
2. Sort those 3 blocks in memory (using quicksort, etc.)
3. Write the sorted data as a **run** (temporary sorted file) to disk.
4. Repeat until all blocks are processed.

📘 Example

Total blocks $b_r = 100$

Memory $M = 3$

Number of initial runs = $\lceil \frac{b_r}{M} \rceil = \lceil \frac{100}{3} \rceil = 34$ runs
Number of initial runs = $\lceil \frac{100}{3} \rceil = 34$ runs

✅ So we get 34 sorted runs (each of 3 blocks, except the last one).

💾 Cost of Run Creation

Each block is:

- **read once**

- **written once**

$\text{Cost} = 2 \times b_r \times \text{Cost} = 2 \times b_r \times \text{Cost} = 2 \times 100 = 200$ block transfers = $2 \times 100 = 200$ block transfers

Step 2: Merge Phase

Now we merge the sorted runs into **bigger sorted runs** — until there's only one final sorted file.
Each merge step is called a **pass**.

How a Merge Works

If memory has MMM blocks:

- We use $M-1$ blocks to hold **input buffers** (runs we are merging).
- We use **1 block** as the **output buffer**.

So we can merge **(M - 1)** runs at a time.

Example:

Memory $M = 3M = 3M = 3$

→ Merge $M-1 = 2M - 1 = 2M - 1 = 2$ runs at a time.

So:

- Pass 1: merge 34 runs into 17 runs
- Pass 2: merge 17 runs into 9 runs
- Pass 3: merge 9 runs into 5 runs
- Pass 4: merge 5 runs into 3 runs
- Pass 5: merge 3 runs into 2 runs
- Pass 6: merge 2 runs into 1 run (final sorted file)

✓ Total = **6 merge passes**

Formula for Number of Passes

Total passes = $\lceil \log_{\frac{M-1}{b_r}} M \rceil$

Using $b_r = 100, M = 3, b_r = 100, M = 3$

$\lceil \log_2(100/3) \rceil = \lceil \log_2 33.3 \rceil = \lceil 5.05 \rceil = 6$

✓ So, 6 merge passes.

Cost of External Sort-Merge

Now we add up all costs.

Run Creation Phase:

$2 \times b_r \times \text{Cost} = 2 \times b_r \times \text{Cost}$

Each Merge Pass:

$2 \times b_r \times \text{Cost} = 2 \times b_r \times \text{Cost}$

(because every block is read and written again)

If there are PPP passes, total merging cost =

$2 \times b_r \times P \times \text{Cost} = 2 \times b_r \times P \times \text{Cost}$

Final Merge Output (no need to write final output to disk)

So, subtract one write pass.

Total Cost Formula

Total Block Transfers = $b_r \times (2 \times \lceil \log_{\frac{M-1}{b_r}} M \rceil + 1)$

Example:

Given:

- $b_r = 100$, $b_r = 100$
- $M = 3$, $M = 3$

Total Block Transfers = $100 \times [2 \times 6 + 1] = 100 \times 13 = 1300$
Total Block Transfers = $100 \times [2 \times 6 + 1] = 100 \times 13 = 1300$

✓ **1300 block transfers**

Total Seeks Formula

Total Seeks = $2 \lceil \frac{b_r}{M} \rceil + b_r (2 \lceil \log_{M-1}(\frac{b_r}{M}) \rceil - 1)$
Total Seeks = $2 \lceil \frac{b_r}{M} \rceil + b_r (2 \lceil \log_{M-1}(\frac{b_r}{M}) \rceil - 1)$

You can plug in the numbers if needed (we'll do an example if you want).

Why Not Just Use Normal Sort?

Because normal (internal) sort works only when **data fits in memory**.

If a table has millions of records, DBMS must use **external sorting** — reading and writing data **in chunks**.

Quick Recap

Step	Name	Description	Disk Cost
1	Run Creation	Sort M blocks at a time, write to disk	$2b_r$
2	Merge Pass	Merge $(M-1)$ runs per pass	$2b_r$ per pass
—	Total	After all passes	$b_r (2 \lceil \log_{M-1}(\frac{b_r}{M}) \rceil + 1) + b_r (2 \lceil \log_{M-1}(\frac{b_r}{M}) \rceil + 1)$

✓ Summary in simple words:

External Sort-Merge = “Sort in pieces, then merge together until one big sorted piece remains.”

Run Creation (External Sort-Merge Phase 1)

Concept

We have:

- A **large relation** on disk — too big to fit in memory.
- Memory of size M blocks.
- Relation (table) with b_r blocks.

We can only process M blocks at a time.

So we read M blocks → sort them → write them back to disk as a **run**.

Example from slide

Let:

- $b_r = 12$, $b_r = 12$ (number of blocks of relation)
- $M = 3$, $M = 3$ (memory size = 3 blocks)

Then:

Number of runs = $\lceil \frac{b_r}{M} \rceil = \lceil \frac{12}{3} \rceil = 4$
Number of runs = $\lceil \frac{b_r}{M} \rceil = \lceil \frac{12}{3} \rceil = 4$

We will create 4 **sorted runs**: R_1, R_2, R_3, R_4

❄ Diagram: “External Sort-Merge (Run Creation)”

Disk (unsorted data)

[Block1][Block2][Block3][Block4][Block5][Block6][Block7][Block8][Block9][Block10][Block11][Block12]

↓ ↓ ↓

(Read 3 blocks into memory)

↓ ↓ ↓

Sort them in memory

↓ ↓ ↓

Write sorted data as Run R1 to disk

Then repeat the process:

Memory

[R4][R5][R6] → sort → write Run R2

[R7][R8][R9] → sort → write Run R3

[R10][R11][R12] → sort → write Run R4

✅ After this phase:

Disk has 4 sorted runs:

R1: sorted(1–3)

R2: sorted(4–6)

R3: sorted(7–9)

R4: sorted(10–12)

❄ Merge Phase (External Sort-Merge Phase 2)

🧠 Goal

Now we have several **sorted runs** (R1, R2, R3, R4).

We must merge them into **one big sorted file**.

We can merge up to **M–1 runs** at a time.

So with $M=3$ $M-1=2$ $M-1=2$:

- $M-1=2$ $M-1=2$ $M-1=2$ → we can merge 2 runs at a time.

❄ Diagram: “Merge pass 1 (2-way merge)”

Memory (3 blocks)

+-----+

| Input Buffer 1 (R1) | → first block of R1

| Input Buffer 2 (R2) | → first block of R2

| Output Buffer | → to store merged output

+-----+

Step-by-step:

1. Read 1 block from **R1** and **R2** into memory.
2. Compare the smallest records from each buffer.
3. Write the smaller one into the **output buffer**.
4. When an input buffer becomes empty → load the next block of that run.
5. When the output buffer is full → write it to disk.

✅ Result:

After merging R1 and R2, you get one larger sorted run.

Do the same for R3 and R4.

After **merge pass 1**:

Input runs: R1, R2, R3, R4

Output runs: R12, R34

✖ Visual of Pass 1 (like your slide diagram)

Memory

```
+-----+ +-----+ +-----+
| R1  | | R2  | | OUT |
+-----+ +-----+ +-----+
      ↓ merge ↓
```

Disk after merge:

R12 (merged from R1, R2)

R34 (merged from R3, R4)

✖ Second Merge Pass

Now we merge **R12** and **R34** to get the final sorted file.

Memory

```
+-----+ +-----+ +-----+
| R12 | | R34 | | OUT |
+-----+ +-----+ +-----+
      ↓ merge ↓
```

Disk after merge:

Rfinal (merged from all)

✅ Finally, we get **one sorted run (Rfinal)** — the fully sorted relation.

📊 Algorithm Analysis Diagrams

Now let's decode the slides showing formulas (and what they mean visually).

✖ Diagram: "Number of Runs"

From the slide:

Total blocks (br) = 100

Memory (M) = 11

Number of runs = $\lceil br / M \rceil = \lceil 100 / 11 \rceil = 10$

Meaning (diagrammatically):

Run Creation Phase:

Each run fits in memory

Memory holds 11 blocks → sort → write → Run R1

Then next 11 blocks → Run R2

...

10 total runs.

✖ Diagram: "Merge Pass 1"

Each merge pass merges **M-1 runs** at a time.

For example:

Memory (M = 11)

Can merge M-1 = 10 runs at a time.

Visual:

Pass 1:

R1..R10 → merged → R'

Pass 2:

R' + remaining runs → merge again

...

until 1 run remains

✳ Diagram: "Passes and Runs"

Each pass reduces the number of runs by a factor of $(M-1)$.

So:

After 1st pass: $= \lceil (br/M) / (M-1) \rceil$ After 1st pass: $= \lceil (br/M) / (M-1) \rceil$

After 2nd pass: $= \lceil (br/M) / (M-1)^2 \rceil$ After 2nd pass: $= \lceil (br/M) / (M-1)^2 \rceil$

Visual:

Pass 0: 10 runs

Pass 1: $10 / (11-1) = 1 \rightarrow$ done (if ≤ 1)

✓ Formula:

Total Passes = $\lceil \log_{(M-1)} (br/M) \rceil$ Total Passes = $\lceil \log_{(M-1)} (br/M) \rceil$

✳ Diagram: "Cost Analysis – Block Transfers"

Each phase (run creation + merge passes) reads & writes every block again.

Each pass:

Read all blocks (br)

Write all blocks (br)

$\rightarrow$ Cost = $2br$ per pass

If total passes = P , total cost = $2br \times P$

But final output isn't written again, so we subtract one write:

✓ Formula:

$BT = br \times [2 \times \lceil \log_{(M-1)} (br/M) \rceil + 1]$ $BT = br \times [2 \times \lceil \log_{(M-1)} (br/M) \rceil + 1]$

✳ Diagram: "Cost Analysis – Number of Seeks"

Each pass needs seeks to read and write runs.

During **run generation**, there's 1 seek per run for read + 1 for write:

$2 \times \lceil br/M \rceil$

Then for merging:

$2br \times \lceil \log_{(M-1)} (br/M) \rceil - br$

✓ Combined:

Total Seeks = $2 \lceil br/M \rceil + br(2 \lceil \log_{(M-1)} (br/M) \rceil - 1)$

✳ Diagram Example Problem

From your slides:

Relation size = 200 blocks

Memory size = 5 blocks

$\rightarrow$ Run Creation:

$\lceil 200/5 \rceil = 40$ runs

$\rightarrow$ Merge Pass 1:

Can merge $5-1=4$ runs at a time

So new runs = $40 / 4 = 10$ runs

$\rightarrow$ Merge Pass 2:

$10 / 4 = 3$ runs (rounded up)

$\rightarrow$ Merge Pass 3:

$3 / 4 = 1$ (final run)

✓ 3 passes total

So the slides show:

Merge 1: 40 → 10 runs

Merge 2: 10 → 3 runs

Merge 3: 3 → 1 run

✅ **Final Cost Example:**

Final merge doesn't need to write to disk, so last write is skipped.

Final Merge Pass: 200 block transfers (read only)

🧠 In Short (Diagram Summary)

Step	Diagram Meaning	What Happens
Run Creation	Chunk → Sort → Save	Sorted runs created
Merge Pass 1	Merge M-1 runs	Fewer, larger runs
Merge Pass 2+	Repeat merging	Until 1 final run
Cost Diagrams	Show log formula	How total I/O grows with size
Example (200,5)	3 merge passes	Final sorted file done

Given:

- Relation size $b_r = 200$ blocks
- Memory size $M = 5$ blocks

1) Number of initial sorted runs (run creation)

We can sort at most M blocks in memory at once, so each initial run contains up to M blocks.

$\text{# initial runs} = \left\lceil \frac{b_r}{M} \right\rceil = \left\lceil \frac{200}{5} \right\rceil = 40$

Explanation / procedure (Run creation):

- Repeatedly: read 5 blocks into memory, sort those 5 blocks in RAM, write the sorted 5-block run back to disk.
- Do this until all 200 blocks are processed → you get 40 runs (most runs are 5 blocks; last run may be ≤5).

ASCII diagram (run creation):

Disk unsorted: [1][2][3]...[200]

Read 5 blocks → sort → write Run R1 (5 blocks)

Read next 5 → sort → write Run R2

...

After run creation:

Disk: R1, R2, R3, ..., R40 (each R_i is sorted internally)

2) Number of runs after first merge pass

In each merge pass you can merge up to $M-1$ runs at once (because 1 block is reserved for output buffer).

Here $M-1 = 4$. So we do a 4-way merge.

$\text{# runs after pass1} = \left\lceil \frac{40}{4} \right\rceil = 10$

Explanation (first merge pass):

- Take runs $R_1..R_4$, merge them into one longer sorted run $R_{12..4}$; do the same for $R_5..R_8$, etc.
- After pass1 you have 10 bigger runs.

ASCII diagram (after pass1):

Input runs: R1 R2 R3 R4 R5 R6 R7 R8 R9 R10 ... R40

Merge groups of 4:

(R1,R2,R3,R4) → S1

(R5,R6,R7,R8) → S2

...

Resulting runs: S1, S2, ..., S10 (10 runs)

3) How many passes until final single run?

Continue merging with 4-way merges:

- After pass1: $40 \rightarrow 10$ runs
- After pass2: $\lceil 10/4 \rceil = 3 \setminus \lceil 10/4 \rceil = 3 \lceil 10/4 \rceil = 3$ runs
- After pass3: $\lceil 3/4 \rceil = 1 \setminus \lceil 3/4 \rceil = 1 \lceil 3/4 \rceil = 1$ run (final)

So **total number of merge passes = 3** (not counting the initial run-creation step which is often called pass 0).

4) Number of block transfers in the final merge pass

Final merge pass = the last pass that produces the single final sorted run (pass3 in our counting). In the usual cost accounting for external sort-merge, the **final merge** only requires **reading** the blocks (to produce output to the parent or to stream to the user) and we *do not count an extra write of the final output back to disk* (because the output can be pipelined to the next operation or returned). Thus:

- In the **final merge pass**, every block is read once: so block transfers = $b_r = 200$ $b_r = 200$.

Why final sorted run is not counted as written to disk:

- Many cost formulas assume the final sorted output is not written back to disk as an extra write (it may be passed directly to the next operator or returned). Writing would double the cost of that pass; by not writing you save that final write. If the system *must* materialize the sorted result on disk, then you would add the write cost (another 200 transfers).

5) (Optional) Total block transfers for the whole external sort

It's useful to compute full I/O cost for all phases (often asked in exercises):

- Run creation (read + write each block): $2 \times b_r = 4002 \times b_r = 4002 \times b_r = 400$ transfers
- Merge pass1 (read + write all blocks): $2 \times b_r = 4002 \times b_r = 4002 \times b_r = 400$
- Merge pass2 (read + write all blocks): $2 \times b_r = 4002 \times b_r = 4002 \times b_r = 400$
- Final merge pass3 (read only, no final write counted): $1 \times b_r = 2001 \times b_r = 2001 \times b_r = 200$

Total block transfers = $400 + 400 + 400 + 200 = 1400$ $400 + 400 + 400 + 200 = 1400$ $400 + 400 + 200 = 1400$.

This matches the formula:

$$\text{Total BT} = b_r \times (2 \times \text{\#passes} + 1) = 200 \times (2 \times 3 + 1) = 200 \times 7 = 1400$$

6) (Optional) Number of seeks — useful if slide asks

Using the slide's seek formula:

$$\text{Total seeks} = 2 \lceil b_r/M \rceil + b_r (2 \lceil \log_{\lceil b_r/M \rceil} M - 1 \rceil - 1) \quad \text{Total seeks} = 2 \lceil \lceil b_r/M \rceil \lceil \log_{\lceil b_r/M \rceil} M \rceil - 1 \rceil + b_r (2 \lceil \log_{\lceil b_r/M \rceil} M \rceil - 1)$$

Compute pieces:

- $\lceil b_r/M \rceil = \lceil 200/5 \rceil = 40$ $\lceil b_r/M \rceil = \lceil 200/5 \rceil = 40$ $\lceil b_r/M \rceil = \lceil 200/5 \rceil = 40$
- $\lceil \log_{\lceil b_r/M \rceil} M \rceil = \lceil \log_{40} 40 \rceil = 1$ $\lceil \log_{\lceil b_r/M \rceil} M \rceil = \lceil \log_{40} 40 \rceil = 1$ $\lceil \log_{\lceil b_r/M \rceil} M \rceil = \lceil \log_{40} 40 \rceil = 1$

So:

$$\text{Total seeks} = 2(40) + 200(2 \times 3 - 1) = 80 + 200(5) = 80 + 1000 = 1080 \quad \text{Total seeks} = 2(40) + 200(2 \times 3 - 1) = 80 + 200(5) = 80 + 1000 = 1080$$

✓ Final concise answers

a) **Initial runs = 40** (run creation: read 5 blocks $\rightarrow$ sort in memory $\rightarrow$ write run).

b) **Runs after first merge pass = 10** (merge 4 runs at a time).

c) **Block transfers in final merge pass = 200** (read-only; final write not counted).

(If asked: **Total block transfers** for whole sort = **1400**; **Total seeks** = **1080**.)

🧠 What Is a Join?

A **join** combines data from **two tables (relations)** based on a **common attribute** (usually a key like id).

🎨 Example

Let's take two small tables:

STUDENT

id name cgpa

1 Asha 3.5

2 Rahim 3.7

3 Rafi 3.2

TAKES

id course-id semester

1 CSE101 Fall

1 CSE102 Spring

2 CSE101 Fall

SQL Query Example

```
SELECT student.id, name, course-id, semester
```

```
FROM student, takes
```

```
WHERE student.id = takes.id;
```

→ This gives us each student with the courses they take.

Relational Algebra

$\pi_{id, name, course-id, semester}(student \bowtie_{student.id=takes.id} takes) \rightarrow \pi_{id, name, course-id, semester}(student \bowtie_{student.id=takes.id} takes)$

Now let's see **how DBMS executes this join internally**.

Nested-Loop Join (NLJ)

This is the **simplest** join algorithm (but not the fastest).

The Basic Idea

For each row in table **r** (outer table):

→ Check every row in table **s** (inner table).

→ If they match, combine them.

Pseudocode

for each tuple **tr** in **r**:

 for each tuple **ts** in **s**:

 if (**tr** and **ts** satisfy join condition):

 output (**tr** ⋈ **ts**)

Visualization

If **r** has 3 rows and **s** has 2 rows:

→ $3 \times 2 = 6$ comparisons!

So if **r** = 1000 and **s** = 500, that's $1000 \times 500 = 500,000$ comparisons 😬

Cost Analysis (Block Transfers)

Let:

- brb_rbr = number of blocks in **r** (outer relation)
- bsb_sbs = number of blocks in **s** (inner relation)

- nrn_rnr = number of tuples in r

Worst Case (one block from each relation in memory)

$Cost = br + (nr \times bs)$ $\text{Cost} = b_r + (n_r \times b_s)$ $Cost = br + (nr \times bs)$

and

$Number\ of\ seeks = br + nr$ $\text{Number of seeks} = b_r + n_r$ $Number\ of\ seeks = br + nr$

Example

Given:

- Student has 20,000 tuples, stored in $br=2000$ $b_r = 2000$ $br=2000$ blocks
- Takes has 40,000 tuples, stored in $bs=400$ $b_s = 400$ $bs=400$ blocks
- Memory can only hold 1 block of each relation

$Cost = nr \times bs + br = 20000 \times 400 + 2000 = 8,000,000 + 2000 = 8,002,000$ $\text{Cost} = n_r \times b_s + b_r = 20000 \times 400 + 2000 = 8,000,000 + 2000 = 8,002,000$ $Cost = nr \times bs + br = 20000 \times 400 + 2000 = 8,000,000 + 2000 = 8,002,000$

$Seeks = nr + br = 20,000 + 2000 = 22,000$ $\text{Seeks} = n_r + b_r = 20,000 + 2000 = 22,000$ $Seeks = nr + br = 20,000 + 2000 = 22,000$

 That's a **huge cost** — NLJ is very slow for large tables!

Best Case (smaller relation fits in memory)

If the smaller relation (say, **takes**) fits fully in memory:

$Cost = bs + br$ $\text{Cost} = b_s + b_r$ $Cost = bs + br$ $Seeks = 2$ $\text{Seeks} = 2$ $Seeks = 2$

So if $bs=400$, $br=2000$ $b_s = 400$, $b_r = 2000$ $bs=400$, $br=2000$:

$Cost = 400 + 2000 = 2400$ $\text{Cost} = 400 + 2000 = 2400$ $Cost = 400 + 2000 = 2400$

 Much faster!

Block Nested-Loop Join (BNLJ)

To make things faster, instead of joining **tuple by tuple**, we join **block by block**.

How It Works

For each **block** in the outer relation rrr :

→ For each **block** in the inner relation sss :

Compare every tuple in both blocks.

Pseudocode

for each block Br in r :

for each block Bs in s :

for each tuple tr in Br :

for each tuple ts in Bs :

if (tr, ts satisfy join condition)

output ($tr \bowtie ts$)

Cost Formula (Worst Case)

$Cost = br \times bs + br$ $\text{Cost} = b_r \times b_s + b_r$ $Cost = br \times bs + br$

and

$Seeks = bs + br$ $\text{Seeks} = b_s + b_r$ $Seeks = bs + br$

Example

Using:

- $br=2000$ $b_r = 2000$ $br=2000$
- $bs=400$ $b_s = 400$ $bs=400$

$\text{Cost} = 2000 \times 400 + 2000 = 800,000 + 2000 = 802,000$
 $\text{Cost} = 2000 \times 400 + 2000 = 800,000 + 2000 = 802,000$
 $\text{Seeks} = 400 + 2000 = 2400$
 $\text{Seeks} = 400 + 2000 = 2400$

✅ Already **10× better** than tuple-level NLJ.

📊 Best Case (smaller table fits in memory)

If one entire relation (say **takes**) fits in memory:

$\text{Cost} = b_s + b_r$
 $\text{Cost} = b_s + b_r$
 $\text{Seeks} = 2$
 $\text{Seeks} = 2$

✅ Same as before: 2400 block transfers and 2 seeks.

⚖️ Comparison: NLJ vs. BNLJ

Type	Inner Loop	Cost (Worst)	Best Case	Comments
NLJ	Tuple by tuple	$n \cdot b_s + b_r$	$b_s + b_r$	Very slow for big tables
BNLJ	Block by block	$b_s + b_r$	$b_s + b_r$	Much faster; uses memory better

🧠 Simple Analogy

Imagine comparing students to course lists:

- **NLJ**: One student checks every course one by one.
- **BNLJ**: A group of students (a block) check a group of courses (another block) together.
→ Less travel, faster work.

✅ Summary

- **Join** = combining two tables.
- **Nested-loop join** checks all pairs (slow).
- **Block nested-loop join** checks blocks (faster).
- **Use indexes or sort-based joins** for very large datasets (we'll see those later).

1) Query we'll use (simple, from your slides)

SQL:

```

SELECT name
FROM instructor JOIN department
  ON instructor.dept_name = department.dept_name
WHERE department.building = 'Watson';

```

Relational-algebra expression tree (top-down):

```

π_name
|
⋈ ( instructor ⋈ department )
/ \
σ_building='Watson' instructor
(department)

```

ASCII tree (labels with operation costs we'll use):

```

π_name
|
(join)
/ \
σ_building instructor
(department)

```

We'll use the numerical costs *from the slides/example* so you can compare materialization vs pipelining numerically:

- Selection σ_{building} (reading the department relation): **1 seek, 20 block transfers**

- Join (joining department result with instructor): **10 seeks, 100 block transfers**
- Projection π_{name} (final projection): **1 seek, 50 block transfers**

Also the slides gave **write costs** (for materializing intermediate results):

- Writing result of $\sigma_{building}$ to disk (materialize): **1 seek, 10 block transfers**
- Writing result of the join (materialize): **1 seek, 50 block transfers**

These numbers are example estimates (like a DBMS catalog would supply). We'll use them to compute totals.

2) Materialization — what is it?

Materialization = evaluate one operation, **write its output to disk** (a temporary relation), then the next operation reads that on disk.

ASCII diagram (materialization flow)

Step A: evaluate $\sigma_{building}$

Read dept -> produce Temp1 (on disk)

(WRITE Temp1 to disk)

Step B: evaluate Join using Temp1 and instructor

Read Temp1 (from disk) + read instructor -> produce Temp2 (on disk)

(WRITE Temp2 to disk)

Step C: evaluate π_{name} using Temp2

Read Temp2 -> produce final result (returned)

Materialization cost calculation (add read costs + write costs)

Sum of *reads* (the operation costs) + *writes* (to store intermediates).

Given the numbers:

- Read costs (operations):
 - σ : 1 seek + 20 blocks
 - Join: 10 seeks + 100 blocks
 - π : 1 seek + 50 blocks

Sum(reads) = (1+10+1) seeks + (20+100+50) blocks = **12 seeks, 170 block transfers**
- Write costs (intermediate writes):
 - After σ -> write Temp1: 1 seek + 10 blocks
 - After Join -> write Temp2: 1 seek + 50 blocks

Sum(writes) = (1+1) seeks + (10+50) blocks = **2 seeks, 60 block transfers**

Total Materialization cost = reads + writes =

- Seeks: 12 + 2 = **14 seeks**
- Block transfers: 170 + 60 = **230 block transfers**

(If the DBMS had to also write the final result to disk, add that write — but usually final result is returned or pipelined to user, so we don't count that write.)

Intuition

Materialization is easy to implement and always possible. But it costs extra I/O because each intermediate result is written to disk and re-read later. That double-handling of data is the expensive part.

3) Pipelining — what is it?

Pipelining = don't write intermediate results to disk. Instead, **pass tuples** directly from one operator to the next as they're produced. Think of an assembly line: one operator produces tuples and immediately hands them to the next operator.

Two models:

- **Producer-driven (push):** child produces tuples and pushes them up to parent (child does work and fills parent buffer).
- **Consumer-driven (pull):** parent requests next tuple from child when it needs it (lazy evaluation).

Both models avoid writing intermediate relations to disk (when operators can stream tuples).

ASCII diagram (pipelining flow)

Evaluate σ_{building} -> as tuples are found, push to Join -> as Join produces joined tuples, push to π_{name} -> output
(No Temp1, No Temp2 on disk)

Cost calculation (pipelining)

We **do not** pay the write cost for intermediate results. So pipelining cost $\approx$ sum of the read costs of operations (since operators still need to read their inputs).

From our numbers:

- Sum(reads) = **12 seeks, 170 block transfers** (same as earlier reads).

So:

Total Pipelining cost = 12 seeks, 170 block transfers

Savings vs materialization:

- Seeks saved = $14 - 12 = 2$ **seeks**
- Block transfers saved = $230 - 170 = 60$ **blocks**

When pipelining actually saves I/O

If the intermediate results are written to disk in materialization, each block typically is written and later read — that's 2 I/O operations per block. Pipelining avoids those writes and re-reads, saving I/O.

4) When is pipelining possible? When not?

Operators that *can* be pipelined easily:

- **Selection (σ)** — produces tuples as it reads.
- **Projection (π)** — can consume tuples and emit immediately.
- **Nested-loop join** (certain variants), **index-nested loop** — can stream output (if inner/outer chosen right).
- **Simple tuple-producing operators** (most relational algebra operators that don't need the whole input).

Operators that are blocking (prevent full pipelining):

These require consuming their entire input (or large portions) before producing any output:

- **Sort** (external sort): must produce sorted runs and then merge — can't output final sorted tuple until sorted order is guaranteed. So it is a blocking operator.
- **Hash-based operators** (e.g., hash-join when you build an in-memory hash table on one input then probe the other): the build phase must be completed before probe can produce full results (though sometimes partial pipelining is possible).
- **Aggregate operators (GROUP BY)** that need to see all tuples for a group (unless grouped and streamed appropriately).

If any operator in the pipeline is blocking, the pipeline is interrupted: materialization (temporary storage) or partial materialization is required at that point.

5) Demand-driven vs Producer-driven (diagrams + pros/cons)

Pull model (Demand-driven)

Top operator (consumer) requests tuple $\rightarrow$ asks child $\rightarrow$ child requests from its child $\rightarrow$... until a leaf scans and returns one tuple

Pros:

- Lazy; only produces what's needed.
- Lower memory footprint sometimes.

Cons:

- Requires operators to maintain state between requests.
- Can cause deep call chains; sometimes less cache-friendly.

Push model (Producer-driven)

Leaf operators read and push tuples up into parent's input buffer; parent consumes when ready

Pros:

- Good for steady-stream processing; can be faster in practice.
- Easier to overlap I/O + CPU.

Cons:

- Needs buffering and backpressure (child must pause if parent is slow).

Both models are used in real systems; DBMS planners choose appropriate model per situation.

6) Worked visualization: materialization vs pipelining with our numbers

Materialization:

σ_{building} (read: 1S,20B) -> write Temp1 (1S,10B)

|
disk Temp1

[Join reads Temp1 + instructor] (read: 10S,100B) -> write Temp2 (1S,50B)

|
disk Temp2

π_{name} reads Temp2] (read: 1S,50B) -> final output

Costs:

- Reads: 1S+10S+1S = 12 seeks; 20+100+50 = 170 blocks
- Writes: 1S+1S = 2 seeks; 10+50 = 60 blocks
- **Total materialize = 14 seeks, 230 block transfers**

Pipelining:

σ_{building} (read 1S,20B) ---> (tuples passed directly)

|
Join (reads instructor too) (10S,100B)

|
 π_{name} (1S,50B)

|
final output

Costs:

- Reads only: **12 seeks, 170 block transfers**

Savings = **2 seeks and 60 block transfers** (we avoid the two intermediate writes and the later reads of those writes).

7) Practical considerations & trade-offs

- **Materialization pros:**
 - Simpler to implement.
 - Robust when operators are blocking.
 - Helpful for reuse of intermediate results (if same intermediate used multiple times).
- **Materialization cons:**
 - Extra I/O (writes + reads) — expensive.
 - Requires disk space for temp files.
- **Pipelining pros:**
 - Less I/O, faster in many cases.
 - Lower latency for first tuples (you can start returning results earlier).

- **Pipelining** cons:
 - Not always possible (blocking ops).
 - Requires operators that support streaming and buffering logic.
 - Can complicate resource scheduling.
-

8) Quick checklist: choose pipelining when...

- No blocking operator (sort, hash-aggregate, etc.) is between producer and consumer.
- Intermediate results are used only once (no need to materialize for reuse).
- Low latency for first results is desired.
- Operators involved support streaming (e.g., nested-loop join variant used).

Materialize when:

- A blocking operator exists or must be inserted (sort for ORDER BY, external sort, some hash joins).
- The intermediate will be reused multiple times by different parents.
- Memory constraints force temporary storage.